Vertical Jump Coach Agent — Progress Report

WatSPEED Agentic AI · Course Project · Sean Tang · due Nov 8, 2026
Code: github.com/seantanger/thp-jump-coach-agent (branch `main`)

For this project, I am building an AI coach for vertical jump training based on the information from John Evans / THP Strength YouTube video transcripts. The work I have done is with the help of Claude Code. The agents use the claude API, and I used the nomic embedding model locally. The first thing I did was collect all of the transcripts from YouTube using yt-dlp, and preprocess the text with regex rules. I then tried three chunking techniques: fixed-size character splitting (LangChain's `RecursiveCharacterTextSplitter`) at two chunk sizes, and my own method that cuts the transcript into ~60-second windows at natural pauses in speech. After embedding the chunks, I made a simple LangGraph loop where the Coach calls a search tool itself (up to 4 searches, rewriting its query if needed) and answers with the video and timestamp behind every point. I tested 3 different search methods on 30 test questions: keyword search (BM25), vector search, and a hybrid of the two. Vector search was the best: with the larger character chunks, the right passage was in the top 5 results for 87% of questions, compared with 47% for keyword search and 67% for hybrid. I have also built a second agent, a Planner, that writes multi-week training programs from reviewed templates, passes them through 8 code checks (e.g. the athlete has the equipment, every cited source was actually retrieved), and waits for my approval before saving.

I went through 4 versions of the Coach prompt, most recently to make answers shorter. For safeguards, a code check rejects any answer that cites a source the search didn't return, a third agent (the Verifier) checks each point against its source in a fresh context, and a code safety check sends messages with red-flag pain symptoms (like a pop in the knee) to a physio instead of coaching them.

Next Steps:

- Connect the Planner and Coach. Right now they are two separate LangGraph graphs: a code router detects plan requests, but the chat doesn't pass them to the Planner yet. I want to combine them into one parent graph with the Coach and Planner as subgraphs. Should routing stay in code or use an LLM?
- Give the Coach access to the current plan and profile, so its answers fit where the athlete is in the program.
- Memory: right now the profile and each plan version are saved as JSON files. Would something like Letta be better?

- Training log: record sessions as done, changed or missed, plus pain and jump results, and use them to decide when the athlete moves to the next stage. (Not sure if this should be like a UI or the user should enter a chat to tell the system that they have finished a workout)
- Plan quality: reading the first plan, sessions need more detail. Accessories should be named exercises ("general lower-body accessory 3×10" doesn't say which one), every lift needs a load (month-1 power cleans have no % because the transcript doesn't give one), and sessions need warm-up and ramp-up sets instead of one weight for every set.
- Optional: MCP server, workout variety in the Planner.

Questions I have:

- How important is evaluation. Could I be evaluating the agent better? So far most of the valuation ideas are what Claude Code is suggesting. I have focused quite a bit on evaluation as Claude Code keeps pushing me to evaluate rather than just trust that its working.

Demos

These call the Claude API (about 4¢ for the Coach question, about 25¢ for a plan) and read the transcript index. The outputs are saved in this notebook. The embedding model used is nomic embedding model in Ollama.

In [1]:
```
import os, sys
from pathlib import Path

ROOT = (Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd())
os.chdir(ROOT); sys.path.insert(0, str(ROOT))
os.environ.setdefault("RETRIEVAL_MODE", "dense")  # dense (vector) search ne
```

Out[1]:
```
'dense'
```

The Coach's LangGraph setup. Two nodes: `coach` (the LLM with tools bound) and `tools` (runs the searches, capped at 4). The graph loops coach → tools → coach until the LLM stops calling tools. `answer_question` (used below) builds this graph, runs it, checks the citations, and sends the answer to the Verifier.

In [2]:
```
# Copy of src/coach/graph.py (the Coach's LangGraph setup). Running it only
import time

from langchain_core.language_models import BaseChatModel
from langchain_core.messages import AIMessage, HumanMessage, SystemMessage
from langgraph.graph import END, START, MessagesState, StateGraph
from langgraph.prebuilt import tools_condition

from src.agent import SEARCH_TOOL, run_capped
```

```
from src.llm import get_llm
from src.prompts import load_prompt
from src.tools import TOOLS
from src.trace import Tracer, usage

SEARCH_CAP = 4


class CoachState(MessagesState):
    search_calls: int


def build_graph(llm: BaseChatModel | None = None, role: str = "coach", trace
                search_cap: int = SEARCH_CAP, require_search: bool = False):
    """Compile the agent graph. Pass `llm` to inject a model (tests, compari
    llm = llm or get_llm(role)
    llm_with_tools = llm.bind_tools(TOOLS)
    system = SystemMessage(content=load_prompt(role))
    tracer = tracer or Tracer()
    tools = {t.name: t for t in TOOLS}

    def coach(state: CoachState) -> dict:
        t0 = time.perf_counter()
        response = llm_with_tools.invoke([system, *state["messages"]])
        if require_search and not response.tool_calls and not state.get("sea
            question = next(m.content for m in state["messages"] if isinstar
            response = AIMessage(content=response.content, usage_metadata=re
                tool_calls=[{"name": SEARCH_TOOL, "args": {"query": questior
        tracer.log("coach", latency_s=round(time.perf_counter() - t0, 3), to
                   tool_calls=[{"name": c["name"], "args": c["args"]} for c
                   text_chars=len(response.text))
        return {"messages": [response]}

    def run_tools(state: CoachState) -> dict:
        out, used = run_capped(state["messages"][-1].tool_calls, tools,
                               {SEARCH_TOOL: state.get("search_calls", 0)},
        return {"messages": out, "search_calls": used[SEARCH_TOOL]}

    graph = StateGraph(CoachState)
    graph.add_node("coach", coach)
    graph.add_node("tools", run_tools)
    graph.add_edge(START, "coach")
    graph.add_conditional_edges("coach", tools_condition, {"tools": "tools",
    graph.add_edge("tools", "coach")
    return graph.compile()
```

1. Ask the Coach a question. The Coach searches the library, answers with citations, then the Verifier checks it. (The two agents are linked through code, not the LangGraph)

In [3]:
```
from src.coach.qa import answer_question

r = answer_question("Should I static stretch before jumping?")
print(r["compact"])
print("\nsearches:", r["queries"])
print("citation check passed:", r["check"]["passed"], "| Verifier:", r["chec
```

```
No, don't static stretch before jumping, because it can reduce force and pow
er output. Warm up dynamically instead. Save static stretching for after the
warm-up, and only if you need it for mobility.

- Skip static stretching before training because it can slightly decrease fo
rce output. Do dynamic flexibility instead. [1]
- Keep the warm-up dynamic. Research has shown static flexibility decreases
power output, and the goal of a jump session is to be highly powerful. Examp
les are bodyweight RDLs, side squats, leg swings and long lunges. [2]
- If you are very immobile, for example tight ankles or hips that limit squa
t depth, static stretches can help. Do them after the dynamic warm-up, not b
efore, e.g. a 2-minute ankle stretch. [3][1]

**Sources**
1. [THP Strength — World Record Vertical Jumper's Exact Warm-Up Routine (50.
5 Inches) @ 00:51](https://www.youtube.com/watch?v=O4piVLyfvSA&t=51s)
2. [John Evans — A Practical 2 Day Per Week Jump System Guaranteed To Get Yo
u Dunking @ 01:42](https://www.youtube.com/watch?v=1sQ-FOxDP5M&t=102s)
3. [John Evans — A Practical 2 Day Per Week Jump System Guaranteed To Get Yo
u Dunking @ 05:42](https://www.youtube.com/watch?v=1sQ-FOxDP5M&t=342s)

searches: ['static stretching in the warm-up before jumping', 'does stretchi
ng before training reduce jump performance, dynamic warm-up']
citation check passed: True | Verifier: pass
```

2. Safety check (code, no LLM). Red-flag symptoms get a physio referral before any agent runs; plan requests are routed to the Planner.

In [4]:

```python
# Copy of src/red_flags.py (referral_kind) and src/router.py. The symptom pa
import re
from typing import Literal
import yaml

RULES = yaml.safe_load(open("config/red_flags.yaml"))


def referral_kind(question):
    # Exclude simple negated symptom lists ("no swelling or bruising").
    # This deliberately handles explicit language, not a clinical diagnosis.
    text = re.sub(
        r"\b(?:no|without|denies?)\s+(?:pain at rest|night pain|back pain|gr
        r"(?:bladder|bowel)(?:/bowel)? changes|(?:sharp(?: or stabbing)?|sta
        r"numbness|tingling|limping|locking|catching|giving way)"
        r"(?:\s*(?:,|or|and)\s*(?:swelling|bruising|numbness|tingling|limpir
        "", question, flags=re.I)
    for kind, key in (("urgent", "urgent_patterns"), ("physio", "referral_pa
        if any(re.search(pattern, text, re.I) for pattern in RULES[key]):
            return kind
    if all(re.search(RULES[key], text, re.I) for key in
           ("duration_pattern", "no_improvement_pattern", "cut_load_pattern"
        return "physio"
    return None


_MONTH = r"(?:jan|feb|mar|apr|may|jun|jul|aug|sep|sept|oct|nov|dec)[a-z]*\.?
```

```
_PROGRAM = re.compile("|".join([
    r"\b(?:plan|program|programme|schedule|mesocycle|macrocycle)s?\b",
    r"\b(?:this|next) (?:week|month)\b",
    r"\bonly (?:train )?\d+ days\b",
    r"\b\d+ days (?:a|per) week\b",
    rf"\b{_MONTH} \d{{1,2}}(?:st|nd|rd|th)?\b",
    r"\bmy (?:dunk )?(?:contest|event|competition|game|meet|tryouts?) is\b",
]), re.IGNORECASE)


def route(message: str) -> Literal["program", "coach"]:
    if referral_kind(message):  # red flags go to the Coach, which answers w
        return "coach"
    return "program" if _PROGRAM.search(message) else "coach"
```

In [5]:
```
for m in ["Should I static stretch before jumping?", "I felt a pop in my kne
          "Make me a plan for the next 12 weeks"]:
    print(f"{m:42} referral={referral_kind(m)}  route={route(m)}")
```

```
Should I static stretch before jumping?    referral=None  route=coach
I felt a pop in my knee this week          referral=physio  route=coach
Make me a plan for the next 12 weeks       referral=None  route=program
```

3. First plan from the Planner. Needs a profile first (`uv run python -m scripts.plan profile`). The Planner drafts a plan, the code checks and Verifier run, then the graph pauses for my approval.

The Planner's LangGraph setup (excerpt of `build_plan_graph` in `src/planner/graph.py`; the node functions are defined above it in the file). `context` loads the profile and current plan, `planner` ⇄ `tools` is the LLM tool loop, `review` runs the 8 code checks and the Verifier (failures go back to the planner, at most 2 retries), `approve` pauses the graph until I approve, reject or revise, and `save` writes a new plan version.

```
graph = StateGraph(PlanState)
for name, node in (("context", context), ("planner", planner),
("tools", run_tools), ("review", review),
                   ("approve", approve), ("save", save)):
    graph.add_node(name, node)
graph.add_edge(START, "context")
graph.add_conditional_edges("context", lambda s: END if
s.get("status") == "needs_profile" else "planner")
graph.add_conditional_edges("planner", after_planner, {"tools":
"tools", "review": "review"})
graph.add_edge("tools", "planner")
graph.add_conditional_edges("review", lambda s:
{"awaiting_approval": "approve", "planning": "planner"}.get(
    s["status"], END))
graph.add_conditional_edges("approve", lambda s: {"approved":
"save", "planning": "planner"}.get(s["status"], END))
graph.add_edge("save", END)
return graph.compile(checkpointer=checkpointer)
```

The athlete profile (me). This is what the Planner reads with its `get_profile` tool. It's stored as `profile.json` and made with `uv run python -m scripts.plan profile`.

In [6]:
```
from src.planner.store import load_profile

print(load_profile().model_dump_json(indent=2))
```


```
{
  "training_age_years": 4.0,
  "lifts": {
    "back_squat_lb": 225.0,
    "half_squat_lb": 245.0,
    "power_clean_lb": 175.0
  },
  "standing_vertical_in": null,
  "approach_vertical_in": 32.0,
  "standing_reach_in": 86.5,
  "height_in": 67.0,
  "days_per_week": 7,
  "available_days": [
    "mon",
    "tue",
    "wed",
    "thu",
    "fri",
    "sat",
    "sun"
  ],
  "session_minutes": 90,
  "equipment": [
    "barbell",
    "rack",
    "platform",
    "box",
    "med ball",
    "safety bar",
    "belt squat"
  ],
  "event": null,
  "goal": "Dunk off two feet. Approach vertical about 31–33 in, standing rea
ch about 7 ft 2–3 in (needs roughly 10 ft 6 in touch). Back squat is a deep
squat; half squat estimated.",
  "knee_pain": {
    "status": "ache",
    "notes": "Knee pain in the past, now manageable."
  },
  "injury_notes": "Two-foot jumper. No sport or other jump load during the w
eek."
}
```


In [7]:
```
from src.planner import run
from src.planner.render import render_proposal
```

```
plan = run.start("new")
print(render_proposal(plan))
```

```
What changes:
  New plan outline
  Month 1: volume
  Month 2: intensification (power and 6–8 rep strength)
  Month 3: absolute strength
  (+5 more)
Why: A new 6-month long-conjugate plan from 2026-11-09 to 2027-04-25, in six
4-week mesocycles, from month 1 volume building to month 6 specificity. Deta
iled days are written for weeks 1 and 2 of month 1 (Nov 9–21). Each week has
three lift days on Mon/Wed/Fri, general days on Tue/Thu, and a Saturday jump
session, with Sunday off.
```

In [8]:
```
plan = run.resume(plan.thread_id, "approve")  # or "reject", or "revise" wit
print(render_proposal(plan))
```

```
Saved as version 1.
```

The saved plan in detail. The 6-month outline (each month cites the chunks it came from), every week written so far (the Planner writes 2 weeks at a time and rolls forward), the chunk IDs behind each exercise, and the videos those IDs point to.

In [9]:
```
from src.coach.citations import link
from src.kb.evidence import chunk_meta
from src.planner.render import render_outline, render_week
from src.planner.store import load_program
from src.planner.templates import TEMPLATE_METHOD

program = load_program()
print(f"Plan version {program.version}: {program.change_summary}\n")
print(render_outline(program, full=True) + "\n")
for week in program.weeks:
    print(render_week(program, week.start, full=True) + "\n")

cites = {c for m in program.mesocycles for c in m.cites} | {
    c for w in program.weeks for d in w.days for s in d.sessions for c in s.
sources = chunk_meta(cites, method=TEMPLATE_METHOD)
print("Sources:")
for c in sorted(sources):
    print(f"  {c}  {link(sources[c])}")
```

Plan version 1: A new 6-month long-conjugate plan from 2026-11-09 to 2027-04-25, in six 4-week mesocycles, from month 1 volume building to month 6 specificity. Detailed days are written for weeks 1 and 2 of month 1 (Nov 9–21). Each week has three lift days on Mon/Wed/Fri, general days on Tue/Thu, and a Saturday jump session, with Sunday off.

Month 1 · Mon Nov 9 – Sun Dec 6 · volume · Build work capacity with general work that looks little like jumping; deload in week 4. [Icml891MLHA@141#0, Icml891MLHA@174#0, Icml891MLHA@209#0]
Month 2 · Mon Dec 7 – Sun Jan 3 · intensification (power and 6–8 rep strength) · Intensify the month-1 volume; Olympic pulls and power cleans for two-foot jumpers; accessories prime month 3. [Icml891MLHA@244#0, Icml891MLHA@282#0, Icml891MLHA@320#0]
Month 3 · Mon Jan 4 – Sun Jan 31 · absolute strength · Heavy squats and heavy clean pulls; eccentric accessories prime month 4. [Icml891MLHA@320#0, Icml891MLHA@354#0, Icml891MLHA@390#0, Icml891MLHA@427#0]
Month 4 · Mon Feb 1 – Sun Feb 28 · strength-speed · Power clean to maxes; accentuated eccentric squats and eccentric split squats; general days get more general. [Icml891MLHA@427#0, Icml891MLHA@463#0, Icml891MLHA@501#0, Icml891MLHA@535#0]
Month 5 · Mon Mar 1 – Sun Mar 28 · speed-strength · Move bars fast (lighter cleans, power snatch, hang cleans); blend a heavy primer with high-velocity squats; start shock loading at low plyo volume. [Icml891MLHA@572#0, Icml891MLHA@609#0, Icml891MLHA@647#0, Icml891MLHA@681#0]
Month 6 · Mon Mar 29 – Sun Apr 25 · specificity · Specificity at its highest; the most plyo intensity and variety of the year; accessories become general and slow. [Icml891MLHA@681#0, Icml891MLHA@718#0, Icml891MLHA@754#0]

Month 1 of 6 · volume · Week 1 of 4

Mon power clean 5×4 · back squat 4×10 @ 65% · general lower-body accessory 3×10
      power clean: Power exercise first; crisp bar speed, moderate load.
      back squat: Higher-rep end, so the lower intensity.
      general lower-body accessory: e.g. sumo deadlift.
      power clean: Icml891MLHA@174#0 Icml891MLHA@209#0
      back squat: Icml891MLHA@141#0 Icml891MLHA@209#0
      general lower-body accessory: Icml891MLHA@141#0 Icml891MLHA@174#0
Tue general circuit (varied movements)
      general circuit (varied movements): Easy bodyweight or bodybuilding circuit with plenty of variety; keep it general.
      general circuit (varied movements): Icml891MLHA@209#0
Wed power clean 5×4 · back squat 4×10 @ 65% · general lower-body accessory 3×10
      power clean: Power exercise first.
      general lower-body accessory: e.g. unilateral work.
      power clean: Icml891MLHA@174#0 Icml891MLHA@209#0
      back squat: Icml891MLHA@141#0 Icml891MLHA@209#0
      general lower-body accessory: Icml891MLHA@141#0 Icml891MLHA@174#0
Thu general circuit (varied movements)
      general circuit (varied movements): Easy circuit; optional and the first thing to drop if tired.
      general circuit (varied movements): Icml891MLHA@209#0
Fri power clean 5×4 · back squat 4×10 @ 65% · general lower-body accessory 3×10
      power clean: Power exercise first.

```
      general lower-body accessory: e.g. sumo deadlift.
      power clean: Icml891MLHA@174#0 Icml891MLHA@209#0
      back squat: Icml891MLHA@141#0 Icml891MLHA@209#0
      general lower-body accessory: Icml891MLHA@141#0 Icml891MLHA@174#0
Sat  jump session warm-up jumps @ 35 min · jump session max-effort jumps @ 5
min
      jump session warm-up jumps: Low-effort jumping inside the 40-minute wi
ndow.
      jump session max-effort jumps: Max-effort jumping; stop if knee pain r
ises above about 2–3/10.
      jump session warm-up jumps: Icml891MLHA@792#0
      jump session max-effort jumps: Icml891MLHA@792#0 72YX7nkYmYE@142#0

Month 1 of 6 · volume · Week 2 of 4

Mon  power clean 5×4 · back squat 5×8 @ 68% · general lower-body accessory 3
×12
      power clean: Power exercise first.
      back squat: Fewer reps, so slightly higher intensity.
      general lower-body accessory: e.g. sumo deadlift.
      power clean: Icml891MLHA@174#0 Icml891MLHA@209#0
      back squat: Icml891MLHA@141#0 Icml891MLHA@209#0
      general lower-body accessory: Icml891MLHA@141#0 Icml891MLHA@174#0
Tue  general circuit (varied movements)
      general circuit (varied movements): Easy circuit with variety.
      general circuit (varied movements): Icml891MLHA@209#0
Wed  power clean 5×4 · back squat 5×8 @ 68% · general lower-body accessory 3
×12
      power clean: Power exercise first.
      general lower-body accessory: e.g. unilateral work.
      power clean: Icml891MLHA@174#0 Icml891MLHA@209#0
      back squat: Icml891MLHA@141#0 Icml891MLHA@209#0
      general lower-body accessory: Icml891MLHA@141#0 Icml891MLHA@174#0
Thu  general circuit (varied movements)
      general circuit (varied movements): Easy circuit; optional.
      general circuit (varied movements): Icml891MLHA@209#0
Fri  power clean 5×4 · back squat 5×8 @ 68% · general lower-body accessory 3
×12
      power clean: Power exercise first.
      general lower-body accessory: e.g. sumo deadlift.
      power clean: Icml891MLHA@174#0 Icml891MLHA@209#0
      back squat: Icml891MLHA@141#0 Icml891MLHA@209#0
      general lower-body accessory: Icml891MLHA@141#0 Icml891MLHA@174#0
Sat  jump session warm-up jumps @ 35 min · jump session max-effort jumps @ 5
min
      jump session warm-up jumps: Low-effort jumping.
      jump session max-effort jumps: Max-effort jumping; stop if knee pain r
ises above about 2–3/10.
      jump session warm-up jumps: Icml891MLHA@792#0
      jump session max-effort jumps: Icml891MLHA@792#0 72YX7nkYmYE@142#0

Sources:
  72YX7nkYmYE@142#0  [THP Strength — You'll Never Get Hurt Again If You Unde
rstand These Principles @ 02:22](https://www.youtube.com/watch?v=72YX7nkYmYE
&t=142s)
  Icml891MLHA@141#0  [John Evans — How to Get a 40" Vertical (Full Blueprin
```

t) @ 02:21](https://www.youtube.com/watch?v=Icml891MLHA&t=141s)
  Icml891MLHA@174#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 02:54](https://www.youtube.com/watch?v=Icml891MLHA&t=174s)
  Icml891MLHA@209#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 03:29](https://www.youtube.com/watch?v=Icml891MLHA&t=209s)
  Icml891MLHA@244#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 04:04](https://www.youtube.com/watch?v=Icml891MLHA&t=244s)
  Icml891MLHA@282#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 04:42](https://www.youtube.com/watch?v=Icml891MLHA&t=282s)
  Icml891MLHA@320#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 05:20](https://www.youtube.com/watch?v=Icml891MLHA&t=320s)
  Icml891MLHA@354#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 05:54](https://www.youtube.com/watch?v=Icml891MLHA&t=354s)
  Icml891MLHA@390#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 06:30](https://www.youtube.com/watch?v=Icml891MLHA&t=390s)
  Icml891MLHA@427#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 07:07](https://www.youtube.com/watch?v=Icml891MLHA&t=427s)
  Icml891MLHA@463#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 07:43](https://www.youtube.com/watch?v=Icml891MLHA&t=463s)
  Icml891MLHA@501#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 08:21](https://www.youtube.com/watch?v=Icml891MLHA&t=501s)
  Icml891MLHA@535#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 08:55](https://www.youtube.com/watch?v=Icml891MLHA&t=535s)
  Icml891MLHA@572#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 09:32](https://www.youtube.com/watch?v=Icml891MLHA&t=572s)
  Icml891MLHA@609#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 10:09](https://www.youtube.com/watch?v=Icml891MLHA&t=609s)
  Icml891MLHA@647#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 10:47](https://www.youtube.com/watch?v=Icml891MLHA&t=647s)
  Icml891MLHA@681#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 11:21](https://www.youtube.com/watch?v=Icml891MLHA&t=681s)
  Icml891MLHA@718#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 11:58](https://www.youtube.com/watch?v=Icml891MLHA&t=718s)
  Icml891MLHA@754#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 12:34](https://www.youtube.com/watch?v=Icml891MLHA&t=754s)
  Icml891MLHA@792#0 [John Evans – How to Get a 40" Vertical (Full Blueprin
t) @ 13:12](https://www.youtube.com/watch?v=Icml891MLHA&t=792s)